Лабораторная работа №9

Сетевое взаимодействие в Python: TCP, клиент-серверная архитектура и прикладные протоколы

1. Цель работы

Освоить создание TCP-сервера и клиента, обмен JSON-сообщениями, обработку нескольких подключений и сетевых ошибок.

2. Постановка индивидуального задания

Вариант 8. Кинотеатр. Основная сущность Movie содержит название фильма, жанр, длительность и возрастной рейтинг. Клиент-серверное приложение поддерживает список, получение, создание, изменение и удаление фильмов, а также специализированную команду поиска фильмов по жанру. Дополнительно реализован asyncio-клиент.

3. Краткие теоретические сведения

TCP обеспечивает надёжный поток байтов, поэтому сообщения приложения разделяются символом новой строки. JSON используется для сериализации команд и ответов. Сервер принимает несколько клиентов и обслуживает каждое соединение в отдельном потоке. Специализированная команда by_genre фильтрует фильмы по жанру. asyncio-клиент демонстрирует асинхронный ввод-вывод.

4. Исходный код программы

Файл async_client.py

```
"""Дополнительный asyncio-клиент для JSON-over-TCP."""
import asyncio, json
HOST="127.0.0.1"; PORT=50508
async def request(command,data=None):
    reader,writer=await asyncio.open_connection(HOST,PORT)
```

```
    message=json.dumps({"command":command,"data":data or {}},ensure_ascii=False)+"\n"
    writer.write(message.encode("utf-8")); await writer.drain()
    line=await reader.readline(); writer.close(); await writer.wait_closed()
    return json.loads(line.decode("utf-8"))
async def main():
    print(json.dumps(await request("list"),ensure_ascii=False,indent=2))
if __name__=="__main__": asyncio.run(main())
```

Файл client.py

```
import socket, json
from protocol import encode_message
HOST="127.0.0.1"; PORT=50508
def request(command,data=None):
    with socket.create_connection((HOST,PORT),timeout=5) as sock:
        sock.sendall(encode_message({"command":command,"data":data or {}}))
        line=sock.makefile("rb").readline()
        if not line: raise ConnectionError("Сервер закрыл соединение без ответа")
        return json.loads(line.decode("utf-8"))
def main():
    while True:
        print("\n=== Кинотеатр: каталог фильмов ===")
        print("1. Все фильмы  2. Фильм по ID  3. Добавить фильм")
        print("4. Изменить фильм  5. Удалить фильм  6. Поиск по жанру  0. Выход")
        choice=input("Выберите пункт: " ).strip()
        try:
            if choice=="1": result=request("list")
            elif choice=="2": result=request("get",{"id":int(input("ID фильма: "))})
            elif choice=="3": result=request("create",{"title":input("Название: "),"genre":input("Жанр:
"),"duration_minutes":int(input("Длительность в минутах: ")),"age_rating":int(input("Возрастной рейтинг: "))})
            elif choice=="4": result=request("update",{"id":int(input("ID фильма: ")),"title":input("Новое название (Enter —
без изменений): "),"genre":input("Новый жанр (Enter — без изменений): " )})
            elif choice=="5": result=request("delete",{"id":int(input("ID фильма: "))})
            elif choice=="6": result=request("by_genre",{"genre":input("Жанр: " )})
            elif choice=="0": break
            else: print("Неизвестный пункт меню."); continue
            print(json.dumps(result,ensure_ascii=False,indent=2))
        except (ValueError,OSError,ConnectionError) as exc: print("Ошибка:",exc)
if __name__=="__main__": main()
```

Файл models.py

```
from dataclasses import dataclass, asdict
@dataclass
class Movie:
    movie_id:int
    title:str
    genre:str
    duration_minutes:int
    age_rating:int=0
    def to_dict(self): return asdict(self)
```

Файл protocol.py

```
import json
MAX_LINE=1024*1024

def encode_message(message): return (json.dumps(message,ensure_ascii=False)+"\n").encode("utf-8")
def decode_message(line):
    if len(line)>MAX_LINE: raise ValueError("Сообщение слишком большое")
    data=json.loads(line.decode("utf-8"))
```

```
    if not isinstance(data,dict) or "command" not in data or not isinstance(data.get("data",{}),dict):
        raise ValueError("Ожидается JSON-объект с command и data")
    return data
```

Файл repository.py

```
class MovieRepository:
    def __init__(self): self.items={}
    def add(self,movie): self.items[movie.movie_id]=movie
    def get(self,movie_id): return self.items.get(int(movie_id))
    def get_all(self): return list(self.items.values())
    def update(self,movie_id,**fields):
        movie=self.get(movie_id)
        if movie is None: return None
        for key,value in fields.items():
            if value is not None and value != '' and key in {"title","genre","duration_minutes","age_rating"}:
setattr(movie,key,value)
        return movie
    def delete(self,movie_id): return self.items.pop(int(movie_id),None)
```

Файл server.py

```
import socket, threading, json
from repository import MovieRepository
from service import MovieService
from protocol import encode_message,decode_message
HOST="127.0.0.1"; PORT=50508
service=MovieService(MovieRepository())

def handle_client(conn,address):
    buffer=b""
    with conn:
        while True:
            chunk=conn.recv(4096)
            if not chunk: break
            buffer+=chunk
            while b"\n" in buffer:
                line,buffer=buffer.split(b"\n",1)
                if not line: continue
                try:
                    request=decode_message(line)
                    result=service.execute(request["command"],request.get("data",{}))
                    response={"status":"ok","data":result}
                except (ValueError,KeyError,TypeError,json.JSONDecodeError) as exc:
                    response={"status":"error","error":str(exc)}
                try: conn.sendall(encode_message(response))
                except OSError: return

def main():
    with socket.socket(socket.AF_INET,socket.SOCK_STREAM) as server:
        server.setsockopt(socket.SOL_SOCKET,socket.SO_REUSEADDR,1)
        server.bind((HOST,PORT)); server.listen()
        print(f"Сервер кинотеатра запущен: {HOST}:{PORT}")
        try:
            while True:
                conn,address=server.accept()
                threading.Thread(target=handle_client,args=(conn,address),daemon=True).start()
        except KeyboardInterrupt: print("\nСервер остановлен.")
if __name__=="__main__": main()
```

Файл service.py

```
from models import Movie
class MovieService:
    def __init__(self,repository):
        self.repository=repository
        if not repository.get_all():
            repository.add(Movie(1,"Интерстеллар","фантастика",169,12))
            repository.add(Movie(2,"Коко","мультфильм",105,6))
            repository.add(Movie(3,"Начало","фантастика",148,12))
            repository.add(Movie(4,"Зелёная книга","драма",130,16))
    def execute(self,command,data):
        if command=="list": return [m.to_dict() for m in self.repository.get_all()]
        if command=="get":
            movie=self.repository.get(data.get("id"))
            if movie is None: raise KeyError("Фильм не найден")
            return movie.to_dict()
        if command=="create":
            title=str(data.get("title","")).strip(); genre=str(data.get("genre","")).strip()
            duration=int(data.get("duration_minutes",0)); age=int(data.get("age_rating",0))
            if not title or not genre or duration<=0 or age<0: raise ValueError("Укажите название, жанр, положительную
длительность и корректный возрастной рейтинг")
            new_id=max((m.movie_id for m in self.repository.get_all()),default=0)+1
            movie=Movie(new_id,title,genre,duration,age); self.repository.add(movie); return movie.to_dict()
        if command=="update":

movie=self.repository.update(data.get("id"),title=data.get("title"),genre=data.get("genre"),duration_minutes=data.get("
duration_minutes"),age_rating=data.get("age_rating"))
            if movie is None: raise KeyError("Фильм не найден")
            if movie.duration_minutes<=0 or movie.age_rating<0: raise ValueError("Некорректная длительность или
возрастной рейтинг")
            return movie.to_dict()
        if command=="delete":
            movie=self.repository.delete(data.get("id"))
            if movie is None: raise KeyError("Фильм не найден")
            return {"deleted":movie.movie_id}
        if command=="by_genre":
            genre=str(data.get("genre","")).strip()
            if not genre: raise ValueError("Укажите жанр для поиска")
            return [m.to_dict() for m in self.repository.get_all() if m.genre.casefold()==genre.casefold()]
        raise ValueError("Неизвестная команда")
```

5. Объяснение работы программы

models.py содержит модель Movie. MovieRepository хранит фильмы, MovieService выполняет CRUD-команды и специализированный поиск по жанру. protocol.py сериализует JSON в newline-delimited сообщения. server.py принимает клиентов в потоках, client.py предоставляет меню, async_client.py выполняет асинхронный запрос списка фильмов.

Для запуска сетевой части откройте два терминала: в первом выполните python server.py, во втором — python client.py. Дополнительный асинхронный клиент запускается командой python async_client.py при работающем сервере.

6. Примеры работы программы в виде снимков экрана

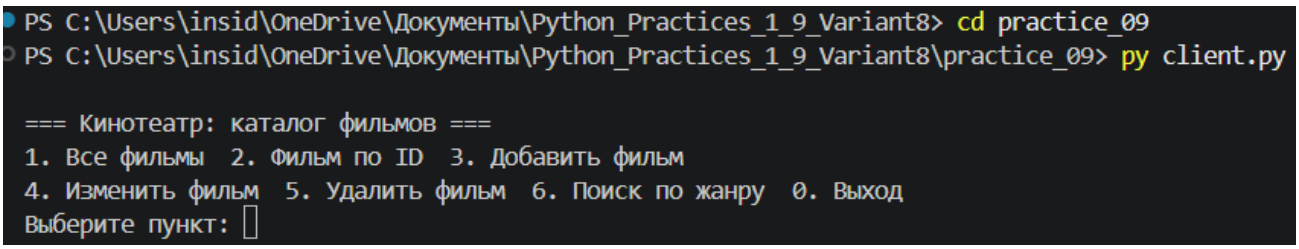


Рисунок 9.1 – Запуск TCP-сервера и успешное подключение клиента.

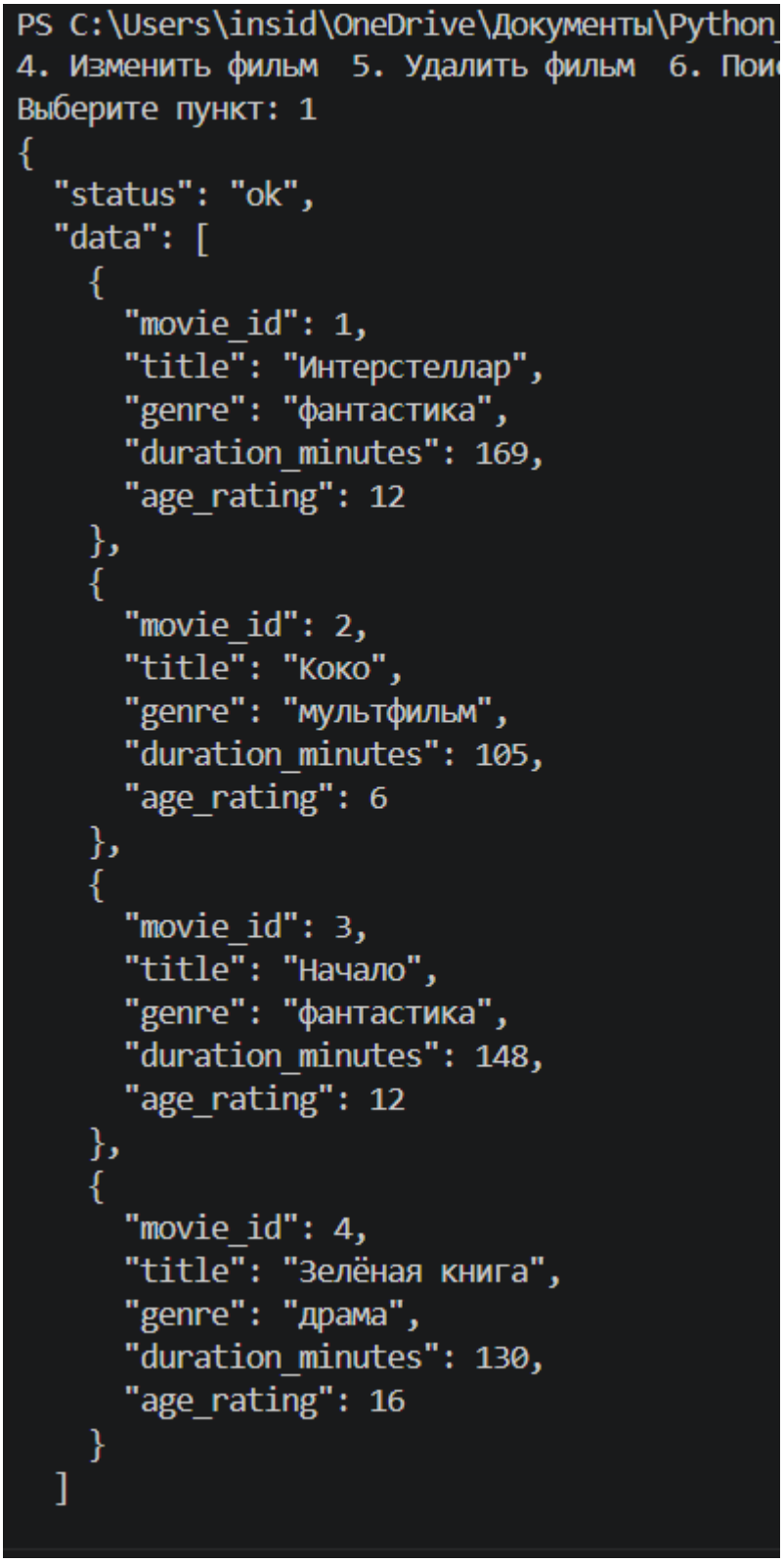


Рисунок 9.2 – Ответ сервера на запрос списка фильмов.

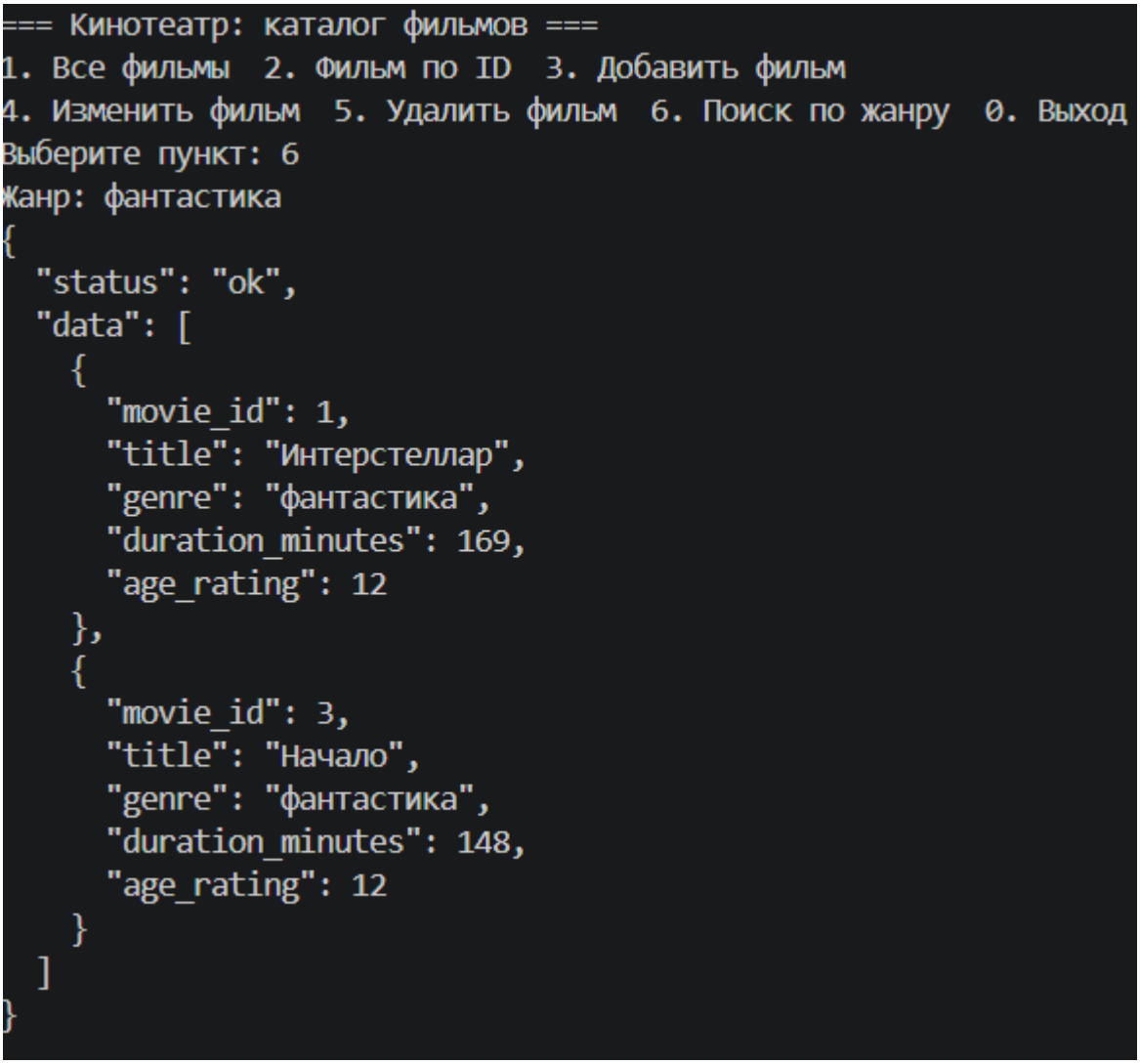


Рисунок 9.3 – Результат специализированного поиска фильмов по жанру.

7. Вывод

В ходе выполнения лабораторной работы были реализованы требования индивидуального задания варианта 8. models.py содержит модель Movie. MovieRepository хранит фильмы, MovieService выполняет CRUD-команды и специализированный поиск по жанру. protocol.py сериализует JSON в newline-delimited сообщения. server.py принимает клиентов в потоках, client.py предоставляет меню, async_client.py выполняет асинхронный запрос списка фильмов. Программа разделена на логические компоненты, содержит проверку типичных ошибок и сопровождается автоматическими тестами.

Выполнил: Куршаков Егор Игоревич